

Assignment 1: Data collection and map view

Introduction

So far, you have developed apps in Android Studio that focused on writing, sending and displaying text, viewing EMF sensor and Motion sensor data and monitoring battery status. In this first assignment we are going to experience app development in the industry, improving an open-source Positioning app developed over the last 3 year by the previous students of this class and interacting with an open-source data collection/positioning API. The project is available on github (<https://github.com/openpositioning/PositionMe>) and we would like for you to follow a gitflow workflow to adapt to the updated data format and sensors requirements, and to add functionality to download and plot the indoor map of a building.

Objectives

For the first assignment, we want to experience a short development cycle, introducing 3 features in the app and testing it. The specific objectives are the following:

- a. **[Introduction]:** Familiarize yourself with the app
- b. **[New feature - update sensor data]:** Update the app to record additional data according to the latest protobuf format specified, this includes among others:
 - Add trajectory name.
 - Add/Update the BLE/WiFi fingerprints:
 - o Verify that WiFi throttling is disabled and the app is always on.
 - o Verify that no repeated fingerprints are used.
 - o Add new data to the scans (WiFi RTT flag, uuid, etc)
 - Add initial position/orientation.
 - Add other sensors data
- c. **[New feature - timestamped markers]:** Add a button during recording session to indicate a test-point
 - Save the timestamp of the pressing
 - Leave a numbered marker on the trajectory in the screen
 - Add the test point to the protobuf

d. [New feature - indoor map display]: Download and display the indoor maps nearby

- Use the current position/observed APs to request nearby indoor maps using the API (<https://openpositioning.org/api/live/floorplan/request>)
- Display on the map the building outline of the available indoor maps/venues,
- if the user clicks on a venue in the map:
 - o Select the venue for data collection,
 - o Show the indoor floorplan and floors
 - o Update the data submission to specify the data collection venue

e. [Testing]: Collect trajectories in the Murchison House

- Record trajectories while you walk on the publicly accessible areas of the building (try to pass at least 2 times over each area/corridor)
- Records some trajectories starting from outside of the building and walking in to at least the ground floor and another floor
- Record trajectories on all stairs and elevators (at least 20 m of a walk before and after the elevation change)
- This data will be used in the next assignment; therefore, we ask for your cooperation in the crowdsourcing.

Some notes about the assignment:

- All work must be implemented in your own repositories forked from the base repository (<https://github.com/openpositioning/PositionMe>)
- Provide your usernames and the forked repository link to your TA. The repository will need to be public at the end of the assignment
- Work on 1 or more (preferred) feature branches ([Gitflow workflow](#)) to divide the work on independent parts
- Make a pull request (use the “across forks” link) to merge your changes to the develop branch of the PositionMe repository, this will leave a record of your code version for review. The pull request must be made on the **12th February 2026, between the 00:00 and 16:00** to avoid code sharing, no commits will be considered after 23:59 11th February 2026.



Figure 1. Areas of interest. Murchison House (red), The Nucleus and Library (amber), Fleeming Jenkin (blue)

Marking Guidelines:

Sr. No.	Criterion	Weightage
1	Demonstration of App Features	35%
2	Code Quality (Reusability, adoption of modular approach, good use of comments, proper alignment, etc.)	30%
3	Documentation (User’s Guide & Programmer’s Guide)	35%

Important Note: You are to develop the application by utilizing the android studio version, SDK version and Gradle version indicated on Learn for consistency (Course Information > Guidelines for Installing Android Studio on your PC).

Demonstration of Application Features Guidelines

These are the guidelines for your application demonstration. You are to upload a 5-10 minute video demonstrating the working of your application. You may choose to make a split-screen video with one side showing the movement of the user and the other side showing the response of the application.

Demo Video Assessment

The demo video assessment marking criteria is as follows:

- Demonstrating key features of the application addressing its key and distinct features.
- Addressing any distinct features that set the application aside from others developed by researchers or available in the market.
- This should not last more than 5-10 minutes. Imagine presenting your application to a customer or an investor.

Documentation (Programmer's & User's Guide)

You need to submit your code through github as a pull request to the PositionMe repository. Documentation for this assignment which would include a user's guide and a programmer's guide must be submitted on the Learn system. The aim of the user guide is to provide instructions to the user on how to use the new features of the app and details on the different features included. You are encouraged to also demonstrate how your app is different from those already in the market (in the User's guide). The aim of the programmer's guide is to explain the functionality and code implementation of your live positioning view improvement to other android programmers so that they could understand the working principles. You are encouraged to also include references of what has been done in literature and how your app is different from that (in the Programmer's guide).

Programmer's guide

Following bullet points should be covered and explained in your document regarding the new features:

1. Application Interface and functionality introduction (20%)
2. Sensor data management (25%)
3. User information used to achieve functionality of the app (25%)
4. Principal methods/listeners implemented in the code (20%)
5. Elaborate on coding style (e.g. commenting, modularity, naming conventions used, etc.) (4%)
6. Extra features (if applicable) and its realization should be explained for marking purpose only (4%)
7. References (if any) (2%)

User's guide

Following bullet points should be covered and explained:

1. Details of the core features that are in the app and what they do (include screenshots) (50%)
2. Instructions on how to use the new features of the app and how to navigate them (include screenshots) (50%)

General rules for the programmer's and user's guide:

1. Each guide should be contained on **two pages** of A4, flexible layout, within reason.
2. Any number of words, but font size not smaller than Times size 10.
3. Each guide cannot exceed two pages of A4, anything beyond will be discarded before marking. No title pages, no appendices will be looked at.
4. Graphs / Charts / Tables can be incorporated – but EVERYTHING has to be contained on two pages of A4.
5. Word processed documents most welcome, equivalent word-processed forms acceptable.
6. The user's guide must be written in layman's terms for users to understand.

Hints on Getting Started

This document should give you a general idea on how to get started. For this assignment you will base your work in an open source data collection app available at <https://github.com/openpositioning/PositionMe>, where you will aim to add features to the project.

In this document, you will learn:

- Project setup
- Adding maps
- Adding markers and lines
- Adding images to the map

Project setup

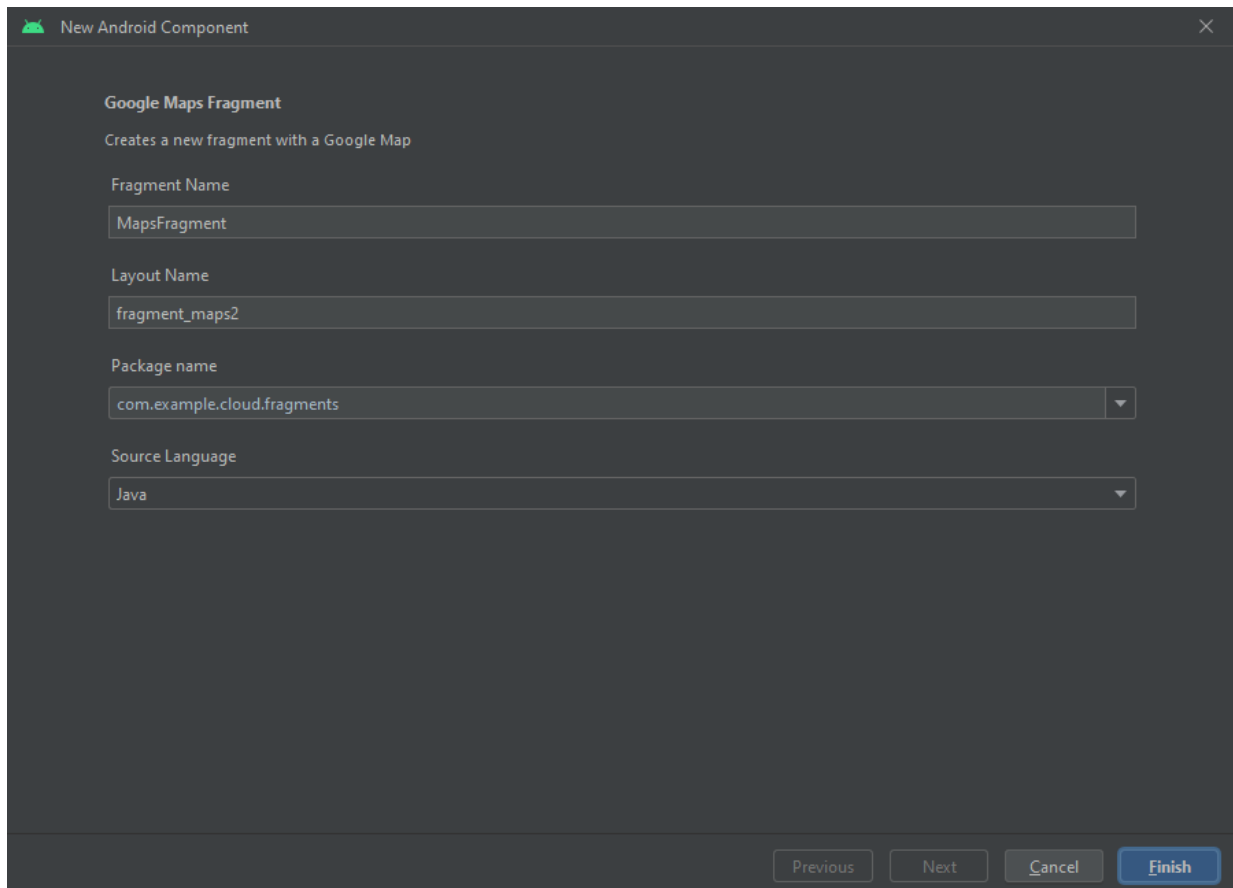
If you haven't already, we recommend you to create an account in github (you will need if you want to push code to the repository) and:

- Clone the repository (git clone <https://github.com/openpositioning/PositionMe.git>).
- Open the project with android studio.
- Obtain an API key for Google Maps (<https://developers.google.com/maps/documentation/javascript/get-api-key>)
- Signup to open positioning and obtain an api_key (<https://openpositioning.org/docs>)
- Modify the secrets.properties file to replace the values of:
 - MAPS_API_KEY: google maps API key
 - OPENPOSITIONING_API_KEY: user's API key from openpositioning.org
 - OPENPOSITIONING_MASTER_KEY: Master API key (ewireless)
- Build your project
- Remember not to push changes with your API keys as you will expose them to the public, ignore changes to the secrets.properties file or remove it from the index for changes detection (git update-index --skip-worktree .\secrets.properties)

Adding maps

There are several options to add a map to an android app (you can also reuse the live view if properly connected), but one of the easiest is through the Maps SDK from google. I recommend you to have a look at the official guide at ([Android-SDK](#)) and in this document we will present some useful features for our task.

Once you have set the google maps API key in the app you can add a **SupportMapFragment** object to the activity that will handle the map. The project already has 2 statically added fragments (fragment_correction and fragment_start_location), you could reuse the start_location to display the position and direction or you could add a new layout fragment. To add a new fragment, you can right click on the layout folder, select New > Fragment > Google Maps Fragment. This will allow you define the names of the fragment, layout and package to use:



The screenshot shows the 'New Android Component' dialog box in Android Studio. The title bar says 'New Android Component' with a close button. The main content area is titled 'Google Maps Fragment' and has a subtitle 'Creates a new fragment with a Google Map'. There are four input fields: 'Fragment Name' with the value 'MapsFragment', 'Layout Name' with the value 'fragment_maps2', 'Package name' with the value 'com.example.cloud.fragments', and 'Source Language' with the value 'Java'. At the bottom, there are four buttons: 'Previous', 'Next', 'Cancel', and 'Finish'.

You will need to modify the layout to define the interface, buttons, textViews, etc, and in the map you will need to add markers and polylines to show the direction and path taken.

Adding markers and lines

To add a marker to the map we just need to add the Latitude and Longitude of the point of interest to the handle of the GoogleMap object. Using the onMapReady callback of the Fragment, we can set a static point like this:

```
@Override
public void onMapReady(GoogleMap googleMap) {
    // Add a marker in Sydney, Australia,
    // and move the map's camera to the same location.
    LatLng sydney = new LatLng(-33.852, 151.211);
    googleMap.addMarker(new MarkerOptions()
        .position(sydney)
        .title("Marker in Sydney"));
    googleMap.moveCamera(CameraUpdateFactory.newLatLng(sydney));
}
```

The addMarker function returns the Marker object, where we can update its position with the setPosition function, and its orientation with the setRotation function. You can explore other properties like visibility, opacity, image, or even set events for dragging, clicking, etc. For a more detailed description of the marker options I recommend the Markers tutorial at: <https://developers.google.com/maps/documentation/android-sdk/marker>

In a similar way, adding a line can be achieved, using Polyline, where we need to define the points of the shape, and add the object to the map like:

```
// Instantiates a new Polyline object and adds points to define a rectangle
PolylineOptions polylineOptions = new PolylineOptions()
    .add(new LatLng(37.35, -122.0))
    .add(new LatLng(37.45, -122.0)) // North of the previous point, but at the same longitude
    .add(new LatLng(37.45, -122.2)) // Same latitude, and 30km to the west
    .add(new LatLng(37.35, -122.2)) // Same longitude, and 16km to the south
    .add(new LatLng(37.35, -122.0)); // Closes the polyline.

// Get back the mutable Polyline
Polyline polyline = map.addPolyline(polylineOptions);
```

To alter the shape of the polyline after it has been added, you can call Polyline.setPoints() and provide a new list of points for the polyline. You can customize the appearance of the polyline both before adding it to the map and after it has been added to the map. See the section on customizing appearances on the tutorial for further details.

Adding images to the map

Adding an image to the map can be easily achieved using Ground overlays, these are rendered with respect to the map orientation and automatically resize with it. To add a GroundOverlay, create a GroundOverlayOptions object that defines both an image and a position. You can optionally specify additional settings that will affect the positioning of the image on the map. Once you've defined the necessary options, pass the object to the GoogleMap.addGroundOverlay() method to add the image to the map. The addGroundOverlay() method returns a GroundOverlay object; you should retain a reference to this object if you want to modify it later. Here is an example to position an image of Newark at a given latitude and longitude, with a width and height in meters:

```
LatLng newarkLatLng = new LatLng(40.714086, -74.228697);

GroundOverlayOptions newarkMap = new GroundOverlayOptions()
    .image(BitmapDescriptorFactory.fromResource(R.drawable.newark_nj_1922))
    .position(newarkLatLng, 8600f, 6500f);
map.addGroundOverlay(newarkMap);
```

If we need to show a different floor of the indoor map, we will need to add an interface to the map and update the image (and boundary) according to the different floors, we can update the image (assuming the same dimensions) with:

```
// Update the GroundOverlay with a new image of the same dimension
imageOverlay.setImage(BitmapDescriptorFactory.fromResource(R.drawable.newark_nj_1922));
```

It is also possible to handle events (click) related to the image, for more information refer to the Ground overlays tutorial at:

<https://developers.google.com/maps/documentation/android-sdk/groundoverlay>